

# **CS 415: Software Design and Development**

## **Project Milestone 0**

Orlando Rodriguez Valdez

Agustin Lemuz-Juarez

RoomSync

Github: <https://github.com/orvaldez/RoomSync>

[orodriguezval@crimson.ua.edu](mailto:orodriguezval@crimson.ua.edu)

[alemuzjuarez@crimson.ua.edu](mailto:alemuzjuarez@crimson.ua.edu)

September 2, 2026

# 1. EXECUTIVE SUMMARY

RoomSync is a web-based household management application designed to help young adults coordinate shared responsibilities with their roommates. Students living together tend to manage household expenses, chores, and other responsibilities through text messages, spreadsheets, payment applications, and informal conversations. These approaches can make it difficult to determine who owes money, what chores have been completed, and what responsibilities are currently assigned to each roommate.

Existing expense-splitting applications are built around temporary groups, such as trips, where each expense is treated as an isolated debt. RoomSync is built for the standing household instead. A household persists across the length of a lease, balances carry forward from month to month rather than resetting, and expenses and chores are managed in the same shared space.

RoomSync will provide the users with a centralized platform where they can create or join a shared household, manage household members, record and split expenses, assign and track chores, and view important household information through a shared dashboard. The web application will focus on providing a simple and accessible experience that allows students to understand their household responsibilities without maintaining multiple complicated separate systems.

The initial MVP will include account creation and authentication, household creation and membership, roommate invitations, shared expense management, expense splitting, settlement recording, chore management, and a household dashboard. Expenses will support equal, custom, and percentage-based splitting. RoomSync will calculate the amount each participant owes based on the selected splitting method. The application records that a debt has been paid but does not process payments. This keeps payment processing, financial account data, and associated security/compliance requirements outside the scope of this project.

The application will be developed as a web application using a JavaScript/TypeScript-based technology stack. The project will follow an Agile development process with incremental working versions delivered at each milestone. GitHub will be used for version control, issue tracking, pull requests, and code review. Automated testing, continuous integration, security, and deployment will be incorporated as the project progresses.

The goal of RoomSync is to provide college students with a practical, centralized solution for managing shared household responsibilities while giving the development team an opportunity to apply software engineering concepts including requirements analysis, system design, implementation, testing, security, version control, CI/CD, deployment, and maintenance.

## 2. TARGET USERS AND PERSONAS

The primary users of RoomSync are college students who live with one or more roommates and share household expenses and responsibilities. These users need a simple way to manage household membership, record expenses and decide how they should be divided, see who owes money, settle outstanding balances, create and assign chores, track what has been completed, and view all of it from a single dashboard. The application is designed primarily for students who have limited time and who rely on their phones and laptops to manage daily responsibilities.

Secondary users are roommates who join a household after it has already been set up, such as a member replacing someone who moved out mid-lease. These users need to join an existing household through an invitation and begin participating without affecting balances recorded before they joined. Additional secondary users may include non-student roommates, such as young professionals sharing an apartment, whose needs are similar to those of the primary users though they may manage a larger number of recurring bills.

The first persona, Sarah Johnson, is a 20-year-old college student who shares an apartment with three roommates. She needs a reliable way to record household expenses, a clear view of who owes money, a way to assign and track chores, and a centralized record of household activity. She also needs to be able to set up the household and invite her roommates to join it. Her goals are to keep household responsibilities organized, avoid repeatedly reminding her roommates about money they owe, and make sure bills are divided correctly. Her constraints are limited time and a preference for her phone and laptop over any system that requires her to maintain a separate spreadsheet.

Sarah often takes responsibility for organizing household expenses. She wants RoomSync to replace informal tracking methods with a centralized system that all roommates can access. Because she is the member who creates the household and manages its membership, Sarah represents the household owner role in the system.

The second persona, Marcus Lee, is a 21-year-old college student who lives in the same apartment as Sarah. He needs a quick way to see what he owes, a list of the chores assigned to him, clear due dates for his responsibilities, and an interface simple enough that it does not require any setup on his part. His goals are to avoid forgetting household responsibilities, quickly determine whether he owes money, and keep up with his assigned chores. His constraints are a busy academic schedule and a habit of not checking group chats, which means he needs the information he cares about presented in one place.

Marcus represents users who may not actively manage household information but still need an easy way to understand and complete their responsibilities.

## 3. PROPOSED APPLICATION

RoomSync provides college roommates with a centralized web application for managing shared household expenses, chores, and membership. The application is intended to reduce the confusion caused by managing household responsibilities across multiple communication and financial applications.

A new user creates a RoomSync account and logs in. The user can then create a household or join an existing household through an invitation. The member who creates a household becomes its owner, and invitations and membership changes are restricted to that role. Once inside a household, the user is presented with a dashboard showing relevant household information, and from that dashboard the user can navigate to expense and chore management.

For example, a roommate who pays a \$120 electricity bill can create an expense, select the roommates participating in it, and choose a splitting method. RoomSync supports three. An equal split divides the total evenly among the selected participants. A custom split lets the user specify the exact amount owed by each participant. A percentage split lets the user specify each participant's percentage of the expense, and RoomSync calculates the corresponding amount. A \$120 expense split equally among four roommates results in a \$30 share for each roommate, so if one roommate paid the entire \$120, RoomSync calculates that the other three each owe that roommate \$30. When a debt is paid, a member records a settlement and RoomSync reduces the balance accordingly. RoomSync does not transfer money between users. It calculates and tracks balances, and roommates settle up through whatever payment method they already use and then record the settlement in the application.

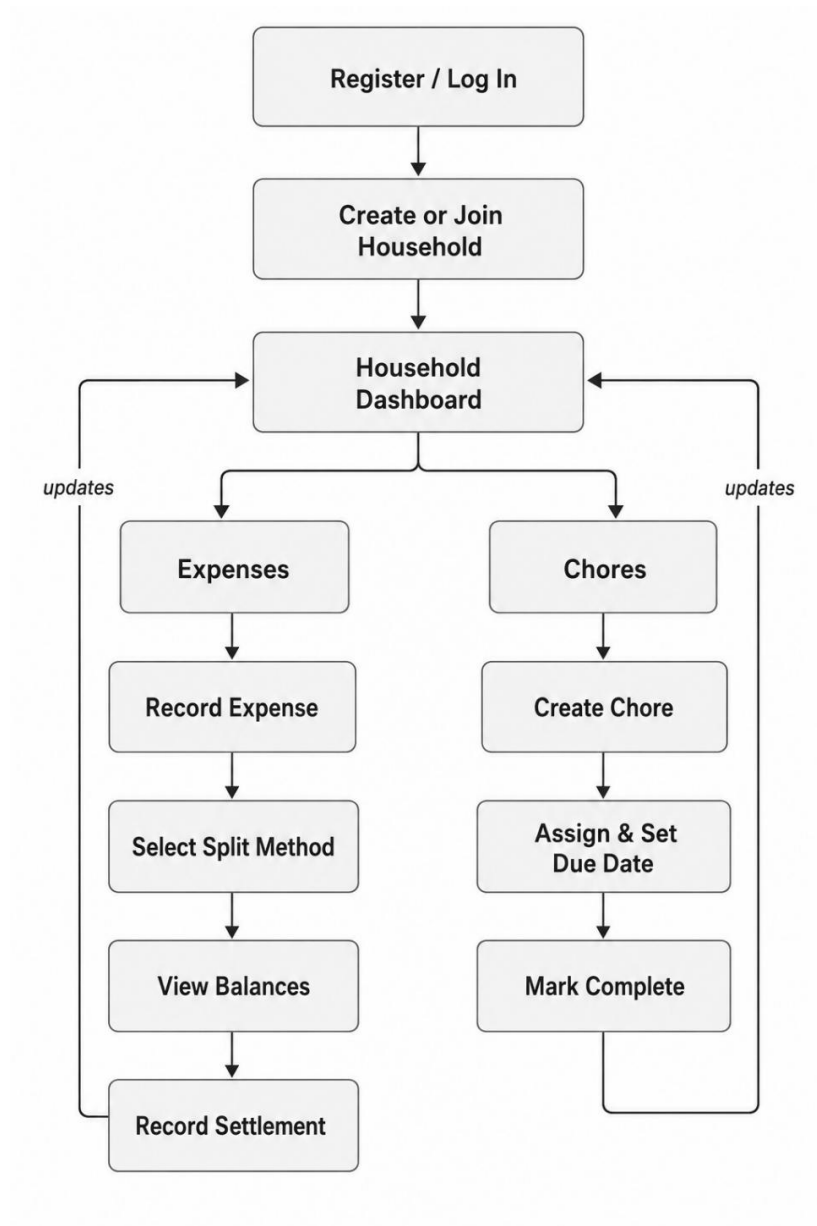
The major features fall into five areas. Account management covers registration, secure login and logout, basic user profile management, and authentication for protected features. Household management covers creating a household, joining one through an invitation, inviting roommates, viewing household members, and managing membership, with invitations and membership changes limited to the household owner. Expense management covers creating expenses, specifying who paid, selecting participants, choosing an equal, custom, or percentage split, calculating individual shares, viewing balances, recording settlements against outstanding balances, and viewing expense and settlement history. Chore management covers creating chores, assigning them to household members, setting due dates, marking them completed, and viewing outstanding and completed chores. The household dashboard brings these together in a single overview showing current household members, recent expenses, amounts the user owes, amounts owed to the user, upcoming chores, and recently completed chores.

RoomSync will be developed as a responsive web application accessible through modern desktop and mobile web browsers. It will not initially be developed as a native iOS or Android application.

A typical end-to-end workflow runs as follows. A user creates a RoomSync account and logs in. The user creates a household and invites roommates, who accept the invitation and join. A roommate records a shared expense, selecting the participants and a splitting method, and

RoomSync calculates each participant's share. Household members view their resulting balances. When a roommate pays what they owe outside the application, a member records the settlement and RoomSync reduces the balance. Separately, a roommate creates and assigns a household chore with a due date, the assigned roommate completes it, and the household dashboard reflects the updated expenses, balances, and chores.

Figure 1. Proposed RoomSync User Workflow



## 4. MVP SCOPE

The following features are required for the Milestone 1 Minimum Viable Product:

| Priority | Feature                        | MVP |
|----------|--------------------------------|-----|
| P0       | User registration              | Yes |
| P0       | User login/logout              | Yes |
| P0       | Create household               | Yes |
| P0       | Join household                 | Yes |
| P0       | Invite roommates               | Yes |
| P0       | View household members         | Yes |
| P0       | Create shared expense          | Yes |
| P0       | Equal expense splitting        | Yes |
| P0       | Custom expense splitting       | Yes |
| P0       | Percentage expense splitting   | Yes |
| P0       | Calculate balances             | Yes |
| P0       | Record settlement              | Yes |
| P0       | Create chore                   | Yes |
| P0       | Assign chore                   | Yes |
| P0       | Set chore due date             | Yes |
| P0       | Mark chore completed           | Yes |
| P0       | Household dashboard            | Yes |
| P1       | Expense and settlement history | Yes |
| P1       | Chore history                  | Yes |

To maintain a realistic semester-long scope, the following features are excluded from the MVP and are not planned for this project: actual payment processing, integration with Venmo, Cash App, Zelle, or banking services, native mobile applications, real-time household chat, automatic bank transaction importing, artificial intelligence-based recommendations, and automatic bill retrieval from utility providers.

If time permits after the required functionality has been completed, potential future features include recurring expenses, recurring chores, email or push notifications, expense categories, expense editing and deletion, member removal, support for multiple households per user, calendar integration, more advanced household analytics, and native mobile application development.

RoomSync focuses on tracking and coordinating household responsibilities rather than processing financial transactions. The application calculates financial obligations and records settlements but does not transfer money between users. The MVP supports one active household per user to reduce unnecessary complexity. RoomSync is delivered as a responsive web application for small households of roughly two to five members, and is not designed for landlords, property managers, or multi-unit properties.

## 5. INITIAL REQUIREMENTS

This section defines the prioritized user stories for RoomSync, acceptance criteria for the high-priority stories, the functional requirements, and the important nonfunctional requirements. Priority P0 indicates a story required for the Milestone 1 minimum viable product. Priority P1 indicates a story planned for the MVP but deferred until the P0 stories are complete.

### **Prioritized user stories and acceptance criteria**

US-01 Account Registration (P0). As a new user, I want to create a RoomSync account so that I can access the application.

- A user can provide the required registration information.
- The system validates the information provided.
- The system prevents duplicate accounts using the same email address.
- Passwords are securely stored using hashing.
- A successfully registered user can log in.

US-02 User Login (P0). As a registered user, I want to log into RoomSync so that I can access my household information.

- A user can enter valid credentials.
- Valid credentials provide access to the application.
- Invalid credentials produce an appropriate error and do not create a session.
- Protected household information cannot be accessed without authentication.

US-03 Create Household (P0). As a user, I want to create a household so that I can manage responsibilities with my roommates.

- An authenticated user can create a household.
- The household receives a unique identifier.
- The creator becomes a member of the household and is assigned the household owner role.
- The household appears on the user's dashboard.

US-04 Invite Roommate (P0). As a household owner, I want to invite another user so that they can join my household.

- The household owner can create an invitation.
- The invitation identifies the household and expires after a set period.
- The recipient can accept the invitation.
- An accepted invitation adds the recipient to the household.
- A member who is not the household owner cannot create invitations.

US-05 Create Expense (P0). As a household member, I want to record a shared expense so that the household can track who owes money.

- The user can enter the expense amount, description, and date.
- The user can identify who paid.
- The user can select participating household members.
- The user can select a splitting method.
- The system rejects amounts that are zero, negative, or not a valid number.
- The system validates the expense before saving it.

US-06 Split Expense (P0). As a household member, I want to split an expense so that each roommate's financial responsibility is calculated correctly.

- Equal splitting divides the total amount evenly among selected participants.
- Custom splitting allows individual amounts to be entered.
- Percentage splitting calculates each participant's share from percentages that sum to 100.
- The sum of participant shares must equal the total expense exactly, including any remainder produced by rounding.
- At least one participant must be included in a split.
- Invalid splits cannot be saved.

US-07 View Balance (P0). As a user, I want to see what I owe and what others owe me so that I understand my household balance.

- The user can view their outstanding balance with each other member.
- The system distinguishes amounts owed by the user from amounts owed to the user.
- Balances reflect all recorded expenses and all recorded settlements.
- A user cannot view balances for a household they do not belong to.

US-08 Record Settlement (P0). As a household member, I want to record that a debt has been paid so that balances reflect money that has already changed hands.

- A member can record a settlement between two members for a specified amount.
- The settlement reduces the outstanding balance between those members.
- A settlement cannot exceed the outstanding balance.
- Settlements remain visible in history after the balance reaches zero.

US-09 Manage Chores (P0). As a household member, I want to create and assign chores so that household responsibilities are distributed among roommates.

- Any household member can create a chore.
- A chore can be assigned to a household member.

- A due date can be specified.
- The assigned user can mark the chore completed.
- Completed chores are distinguishable from outstanding chores.

US-10 View Dashboard (P0). As a household member, I want to view a household dashboard so that I can quickly understand my responsibilities.

- The dashboard displays current household members.
- The dashboard displays the user's outstanding balances.
- The dashboard displays upcoming chores assigned to the user.
- The dashboard displays recent household activity.

US-11 View History (P1). As a household member, I want to view past expenses, settlements, and chores so that I can check what a charge was for, confirm that a debt was paid, or confirm that a task was completed.

- The user can view a list of past expenses for the household.
- The user can view a list of recorded settlements for the household.
- The user can view a list of completed chores for the household.

## **Functional requirements**

- FR-01: The system shall allow users to register accounts.
- FR-02: The system shall authenticate registered users.
- FR-03: The system shall prevent unauthenticated users from accessing protected household information.
- FR-04: The system shall allow authenticated users to create or join a household.
- FR-05: The system shall assign the household owner role to the user who creates a household.
- FR-06: The system shall restrict invitations and membership changes to the household owner.
- FR-07: The system shall allow household members to create shared expenses.
- FR-08: The system shall support equal, custom, and percentage-based expense splitting.
- FR-09: The system shall validate that expense shares equal the total expense amount, and that percentage-based splits use percentages summing to 100.
- FR-10: The system shall calculate the amount each participant owes.
- FR-11: The system shall allow household members to record settlements against outstanding balances and shall update balances accordingly.
- FR-12: The system shall maintain expense and settlement history.
- FR-13: The system shall allow household members to create chores.
- FR-14: The system shall allow chores to be assigned to household members.
- FR-15: The system shall allow assigned users to mark chores as completed.
- FR-16: The system shall display household information through a dashboard.
- FR-17: The system shall store application data in a persistent database.

- FR-18: The system shall store all monetary values as integer cents.

### **Nonfunctional requirements**

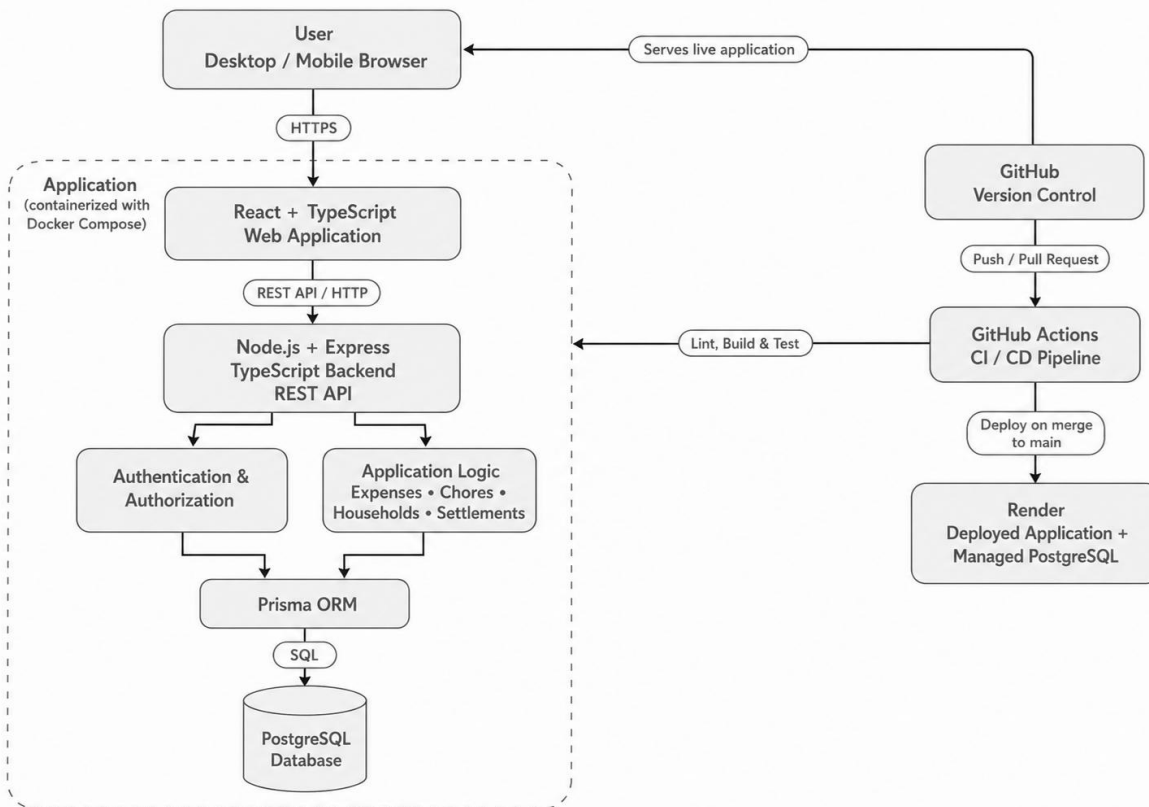
- NFR-01 Usability: RoomSync shall provide a straightforward interface that allows a typical user to perform common actions without requiring technical knowledge. A member shall be able to record a typical expense in no more than four interactions from the household dashboard.
- NFR-02 Performance: Common operations such as loading the dashboard or retrieving household expenses shall complete in under two seconds for a household of five members with 500 recorded expenses.
- NFR-03 Reliability: The system shall preserve valid household, expense, chore, and settlement information and shall prevent invalid data from being stored. Balances shall be derived from stored expenses and settlements rather than from a cached total, so that balances cannot drift from the underlying records.
- NFR-04 Accessibility: The interface shall be operable by keyboard alone, shall meet WCAG AA contrast ratios, and shall label all form inputs. The application shall use readable text, clear navigation, and semantic interface elements where practical.
- NFR-05 Responsiveness: The application shall be fully usable at a viewport width of 375 pixels.
- NFR-06 Privacy: Users shall only be able to view household information for households to which they belong. Personal account information shall not be unnecessarily exposed to other users, and the system shall collect only the data required to operate.
- NFR-07 Security: Passwords shall never be stored as plain text and shall be hashed using a modern password hashing algorithm with a per-user salt. Protected routes shall require authentication, and authorization checks shall prevent users from reading or modifying resources belonging to households they do not belong to. All user-provided data shall be validated on the server, and database access shall use parameterized queries.

## **6. INITIAL DESIGN AND ARCHITECTURE**

RoomSync will use a client-server architecture. The client is a React single-page application running in the user's browser, responsible for presenting information and collecting input. The server is a Node.js and Express REST API containing the authentication and authorization layer and the application logic for households, expenses, settlements, and chores. The database is a PostgreSQL instance accessed through Prisma, providing persistent storage for users, households, memberships, expenses, expense shares, settlements, and chores. The external services are GitHub for version control, GitHub Actions for continuous integration and deployment, and Render for hosting the deployed application and its managed PostgreSQL

instance. RoomSync does not depend on any third-party runtime API, which keeps the application free of external availability and licensing risk.

Figure 2. Proposed RoomSync System Architecture



## Proposed Technology Stack

| Component         | Proposed Technology                                       | Purpose                                       |
|-------------------|-----------------------------------------------------------|-----------------------------------------------|
| Frontend          | React                                                     | Web user interface                            |
| Frontend language | TypeScript                                                | Type-safe client-side development             |
| Backend           | Node.js and Express                                       | Server and REST API                           |
| Backend language  | TypeScript                                                | Consistent development language               |
| Database          | PostgreSQL                                                | Persistent relational data storage            |
| ORM               | Prisma                                                    | Database access and schema management         |
| Authentication    | Session-based authentication with bcrypt password hashing | User authentication and password storage      |
| Containerization  | Docker and Docker Compose                                 | Reproducible local development and deployment |

|                 |                               |                                                 |
|-----------------|-------------------------------|-------------------------------------------------|
| Version control | Git and GitHub                | Source control and collaboration                |
| CI/CD           | GitHub Actions                | Automated build, test, and deployment workflows |
| Testing         | Vitest or Jest with Supertest | Unit and API testing                            |
| Deployment      | Render                        | Hosting the production application and database |

## Technology Justification

React and TypeScript are proposed for the frontend because they provide a component-based approach to building a modern web interface while TypeScript provides static type checking. Node.js and Express provide a lightweight backend environment that works well with TypeScript development. Using TypeScript on both the frontend and backend reduces context switching and allows the team to maintain a consistent programming language across the application, which matters for a two-person team where both members work across the full stack.

PostgreSQL is appropriate because RoomSync contains multiple related entities, including users, households, memberships, expenses, expense shares, settlements, and chores. A relational database provides the constraints and relationships needed to maintain data integrity, particularly for balance calculations that depend on the correctness of every expense share. A document database was considered but rejected, because RoomSync's core operations are relational and depend on integrity across these entities rather than on flexible document structure. Prisma provides an abstraction layer between the backend and PostgreSQL, allows the schema to be defined and versioned in code, and generates parameterized queries, which supports the input handling described in NFR-07.

Docker and Docker Compose are included from the start rather than added late, because the course requires the application to be runnable by a TA on an independent machine and requires containerized services at Milestone 3. Running PostgreSQL as a container locally means the same database configuration is used in development and production. A managed backend platform such as Firebase was considered but rejected, because it would make a reproducible local setup difficult and would obscure the server-side authorization logic the project is intended to demonstrate.

Render is proposed for deployment because it deploys containerized applications directly from a GitHub repository, provisions managed PostgreSQL, and redeploys automatically on push, which supports the CD pipeline required at Milestone 3 without significant infrastructure configuration. Because the application is containerized and all configuration is supplied through environment variables, the deployment target remains portable, and the team could move to a comparable provider such as Railway or Fly.io without changing application code.

GitHub will be used for version control, pull requests, issue tracking, and CI/CD integration through GitHub Actions.

## 7. DATA AND EXTERNAL RESOURCES

The initial RoomSync data model is expected to contain the following entities.

- User. Stores account information for each RoomSync user. Potential attributes: user ID, name, email, password hash, account creation date.
- Household. Represents a shared living group. Potential attributes: household ID, household name, creation date.
- Membership. Associates users with households. Potential attributes: membership ID, user ID, household ID, role, join date. The role attribute distinguishes the household owner from ordinary members and is the basis for the authorization checks described in NFR-07.
- Invitation. Represents an invitation for a user to join a household. Potential attributes: invitation ID, household ID, invite token, invitation status, creation date, expiration information.
- Expense. Represents a household expense. Potential attributes: expense ID, household ID, description, total amount, user who paid, splitting method, creation date. Monetary amounts are stored as integer cents in accordance with FR-18.
- Expense Share. Represents an individual user's responsibility for an expense. Potential attributes: expense share ID, expense ID, user ID, amount owed as integer cents, percentage where applicable.
- Settlement. Represents a payment made outside the application to reduce an outstanding balance. Potential attributes: settlement ID, household ID, paying user, receiving user, amount as integer cents, date, optional note.
- Chore. Represents a household responsibility. Potential attributes: chore ID, household ID, title, description, assigned user, due date, completion status, creation date.

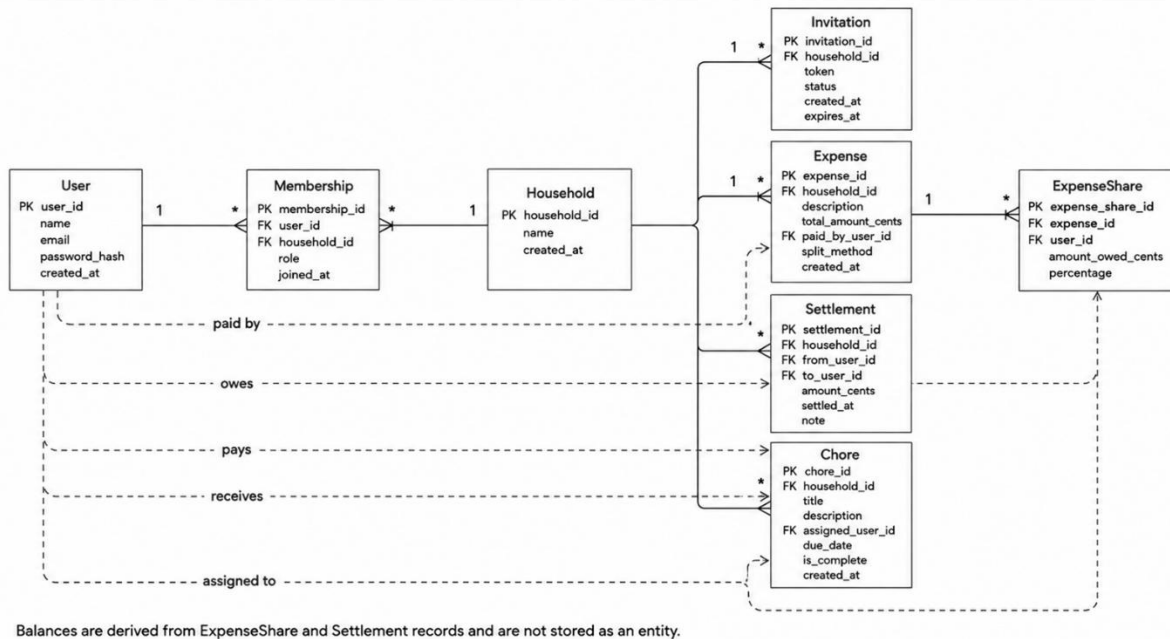
### Data relationships

The primary relationships are expected to include the following:

- A user belongs to a household through a membership, and a household can contain multiple members.
- A household can contain multiple expenses, and an expense can contain multiple expense shares, each of which belongs to a user.
- A household can contain multiple settlements, and each settlement references two users who are both members of that household.
- A household can contain multiple chores, and a chore can be assigned to a household member.
- A household can have multiple invitations.

- Balances are not stored as an entity; they are derived from expense shares and settlements, in accordance with NFR-03.

Figure 3. Initial RoomSync Entity Relationship Diagram



## External resources

RoomSync does not require any external dataset for the MVP. All application data is created by users within the application itself. The external resources the project depends on are development and operational services rather than data sources.

The cloud services are GitHub for version control and issue tracking, GitHub Actions for continuous integration and deployment, and Render for hosting the deployed application and its managed PostgreSQL instance. The primary libraries and frameworks are React for the web client, Node.js with Express for the server, TypeScript across both, Prisma as the ORM, bcrypt for password hashing, Vitest or Jest with Supertest for unit and API testing, and Docker with Docker Compose for containerized local development. RoomSync requires no special hardware or devices beyond the computers used for development and a standard web browser for access.

## Access, licensing, privacy, and cost concerns

RoomSync does not depend on any third-party runtime API. The application does not consume an external service to function, so the project carries no API key management, request

quotas, rate limits, or vendor availability risk. This was a deliberate design choice and it removes a category of risk that would otherwise be outside the team's control.

The proposed libraries and frameworks are open source under permissive licenses, primarily MIT and Apache 2.0, and are compatible with use in an academic project. Any additional dependency added during development will be reviewed for license compatibility before it is included.

RoomSync collects only the personal information required to operate: a name, an email address, and a hashed password. The application does not store banking credentials, payment account information, or any financial identifiers, and it does not process financial transactions. Expense descriptions are user-supplied free text and are visible to all members of the household in which they are recorded, so household data is scoped to its household and is never exposed across households. This scoping is enforced on the server for every request, as described in NFR-06 and NFR-07.

The project is expected to operate within free or educational tiers. GitHub and GitHub Actions are free for this scale of use, and Render offers a free tier that covers both the application and a PostgreSQL instance. The known limitation of the free tier is that idle instances are suspended and take several seconds to restart on the next request, which affects initial load time during a demonstration but does not affect functionality. If the free database tier proves insufficient, the team will evaluate a low-cost paid tier or an alternative provider, which remains straightforward because the application is containerized and configured entirely through environment variables.

## 8. AGILE DEVELOPMENT PROCESS

### **Sprint Length**

The team will work in two-week sprints, for approximately six sprints across the semester. Sprint boundaries will be aligned so that each project milestone falls at the end of a sprint. Each sprint contains planning, implementation, testing, review, and a short retrospective.

### **Backlog and Issue Tracking**

GitHub Issues will be used to manage the product backlog and all development tasks. Issues will represent features, bugs, testing tasks, documentation tasks, security improvements, and deployment tasks. Each issue is labeled with its type and its priority, using the P0 and P1 priorities defined in the requirements section, and is assigned to the team member who owns that area of the project. Issues are prioritized according to their importance to the MVP and to the milestone in progress, and the backlog is reviewed at the start of each sprint.

## **Branching strategy**

The team will maintain a protected main branch containing stable code. No work is committed directly to main. All work is done on branches created from main and merged back through pull requests.

Branches are named by type and short description: feature/<short-description> for new functionality, fix/<short-description> for defect repairs, test/<short-description> for test work, and docs/<short-description> for documentation. Example branches include feature/authentication, feature/expense-splitting, feature/chores, and feature/dashboard.

Commit messages follow the Conventional Commits convention, using prefixes such as feat:, fix:, test:, docs:, and refactor: followed by a short description of the change. This keeps the commit history readable and supports the changelog required at Milestone 2.

## **Pull requests and code review**

Each feature is developed on its own branch and submitted through a pull request that references the issue it resolves. Every pull request must be reviewed and approved by the other team member before it is merged. No pull request is merged by its own author without a review, and there are no exceptions to this rule.

Reviews consider correctness, readability, security, adequacy of tests, compliance with the stated requirements, and maintainability. Because both members work across the full stack and review all of each other's work, no single member becomes the sole owner of a major part of the application. Beginning at Milestone 3, the continuous integration pipeline will block merges when automated checks fail.

## **Definition of Done**

A task is considered complete when all of the following are true:

- The required functionality has been implemented and meets the acceptance criteria of its user story.
- The implementation follows the project's coding standards.
- Relevant unit or integration tests have been created or updated and pass.
- Automated checks in the continuous integration pipeline pass.
- The change has been reviewed and approved by the other team member.
- The change integrates successfully with the existing application and does not break existing functionality.
- No known critical defects remain.
- Related documentation, including the README where applicable, has been updated.

## **Communication**

Day-to-day communication takes place through a group messaging channel used for scheduling, quick questions, and blockers. All discussion specific to code takes place in GitHub, through issue comments and pull request reviews, so that decisions are recorded alongside the work they affect. The team holds a check-in at the start of each week to review progress, identify blockers, and confirm what each member is working on, and a short retrospective at the end of each sprint to identify what should change in the next one.

## 9. MILESTONE PLAN

Every milestone from Milestone 1 onward will contain a runnable version of RoomSync. At each milestone the repository will contain the complete source code for that version, setup and dependency instructions, environment variable documentation with a safe example file, database migration and seed instructions, demonstration accounts, and a short verification guide describing what the TA should run and check. The TA will be able to clone the repository and run the application on an independent machine. Once deployment is in place, the public URL will be provided in addition to the local instructions. Each milestone will be tagged in Git as milestone-0, milestone-1, milestone-2, and milestone-3, with a final v1.0 release tag at the end of the project.

| Milestone   | Main goal | Planned deliverables                                                                                                                                                                   |
|-------------|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Milestone 0 | Propose   | Proposal PDF, initial architecture diagram, prioritized backlog, shared GitHub repository, initial README, contribution statements, tag milestone-0                                    |
| Milestone 1 | Build     | Working end-to-end MVP, design and requirements artifacts, setup and verification instructions, MVP demonstration, contribution statements, tag milestone-1                            |
| Milestone 2 | Test      | Expanded working application, test and coverage evidence, code review evidence, security evidence, updated setup instructions, demonstration, contribution statements, tag milestone-2 |
| Milestone 3 | Deploy    | Complete deployed application at a public URL, final report and technical                                                                                                              |

|  |  |                                                                                                                  |
|--|--|------------------------------------------------------------------------------------------------------------------|
|  |  | evidence, reproducible local setup, live demonstration, final contribution statements, tags milestone-3 and v1.0 |
|--|--|------------------------------------------------------------------------------------------------------------------|

### **Milestone 1: Build the MVP**

The team will implement the core RoomSync application as a working end-to-end system. Functionality will include account registration and login, household creation and joining, invitations, expense creation and splitting across all three splitting methods, balance calculation, settlement recording, chore creation and assignment, and the household dashboard.

Alongside the working application, Milestone 1 includes the design and requirements artifacts: an updated product brief and MVP scope, the chosen software process model with justification, a prioritized backlog and Definition of Done, use cases for the main backlog items, an analysis model showing the important data and behavior, design classes with an API contract and module boundary diagram, an Architecture Decision Record, component designs for at least two important use cases, UX wireframes covering the multi-screen interface, responsive design considerations, and one or two design patterns with justification.

### **Milestone 2: Test, improve, and secure**

The team will expand the application with additional completed features and raise its quality. Feature work will include the P1 history views and selected items from the future features list, such as expense editing and deletion and member removal.

Quality and security work will include unit tests with a coverage report and a justified coverage target, integration tests for the main user flows, cross-viewport testing evidence, a quality assurance checklist, evidence of peer code review through pull requests, a threat model covering the main attack surfaces, a review of the relevant OWASP Top 10 categories, at least one implemented security control with before and after evidence, documented branching and commit conventions, a changelog, and at least two tagged releases. Continuous integration will run automated linting and tests on every pull request.

### **Milestone 3: Deploy**

The final milestone focuses on production readiness. Work will include containerized services with a working local setup, a continuous integration pipeline that blocks merges when checks fail, a continuous deployment pipeline, and a live application at a public URL.

Milestone 3 also includes a metrics report covering coverage trends, a code quality indicator, and deployment frequency; one feature or refactor completed with a disclosed AI-assisted workflow along with a record of what was asked of the AI and what was accepted, edited, rejected, and why; a risk register and maintenance and support plan; a polished README; a live

demonstration; and a semester retrospective. Any major concepts not completed at Milestone 1 or 2 will be finished here.

## Schedule

The project runs across approximately six two-week sprints, with milestone dates following the course semester schedule.

| Period   | Focus                                                                      | Milestone                    |
|----------|----------------------------------------------------------------------------|------------------------------|
| Sprint 1 | Repository setup, schema, authentication, household creation               |                              |
| Sprint 2 | Invitations, expenses, splitting, balances, settlements, chores, dashboard | Milestone 1 at end of sprint |
| Sprint 3 | Automated testing, validation, error handling, history views               |                              |
| Sprint 4 | Threat model, security control, OWASP review, CI, additional features      | Milestone 2 at end of sprint |
| Sprint 5 | Containerization, deployment pipeline, production deployment               |                              |
| Sprint 6 | Final features, documentation, metrics, retrospective, demonstration       | Milestone 3 at end of sprint |

## 10. TEAM RESPONSIBILITIES

Both members will work across the full stack rather than splitting the project into a frontend and backend role. Each feature is owned by one member and built end to end, from the database schema through the API to the interface. It is then reviewed by the other member before it merges. This keeps both members familiar with the entire codebase, avoids a situation where one person is blocked waiting on the other, and produces a clear record of who built what.

Coding, design, testing, documentation, review, deployment, and presentation work are divided as follows. For the MVP, Orlando owns accounts, login, and session handling, along with balance calculation and settlement recording. Agustin owns household creation, invitations, and membership, along with expense entry and split logic. Beyond feature code, Agustin owns branching and commit conventions and continuous integration configuration, as well as architecture diagrams, design documentation, and deployment and containerization. Orlando owns the database schema, migrations, and seed data, the README, setup instructions, and

verification guide, and the test plan, coverage reporting, and quality assurance checklist. Milestone submission assembly and the contribution statements are shared.

Code review is shared without exception. Both members review and approve every pull request that the other opens, and no branch merges into main without a review. Both members write unit tests for the code they own and integration tests for the flows they build. Presentation work is also shared: for the live demonstration at Milestone 3, Orlando will present the expense, balance, and settlement flow and Agustin will present the architecture and deployment.

Ownership is not permanent. Assignments are reviewed at the start of each milestone and adjusted so that neither member spends the whole semester on the same kind of work. Both members are expected to contribute roughly half of the total effort across coding, testing, documentation, review, deployment, and presentation preparation. If the workload becomes uneven, the team will address it at the next sprint boundary rather than at the end of the semester.

## 11. RISKS AND MITIGATION

| Risk                                        | Category   | Likelihood | Impact | Mitigation                                                                                                                                                                                   |
|---------------------------------------------|------------|------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Project scope becomes too large             | Scheduling | Medium     | High   | Prioritize MVP requirements and defer optional features until required functionality is complete                                                                                             |
| Team falls behind schedule                  | Scheduling | Medium     | High   | Use two-week sprints aligned to milestones and track all work through GitHub Issues                                                                                                          |
| One member becomes unavailable for a period | Team       | Medium     | High   | Both members work full-stack and review all of each other's code, so either can pick up the other's area; ownership is reassigned at the next sprint boundary rather than letting work stall |
| Unequal workload between members            | Team       | Medium     | High   | Track contributions through GitHub Issues and pull requests, and address imbalance at the next sprint boundary                                                                               |
| Database design problems                    | Technical  | Medium     | Medium | Design entity relationships early, review the schema before implementation, and manage schema changes through Prisma migrations                                                              |
| Incorrect balance                           | Data       | Medium     | High   | Store all monetary values as integer cents per FR-18, derive balances from stored records                                                                                                    |

|                                                  |            |        |        |                                                                                                                                                           |
|--------------------------------------------------|------------|--------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| calculations from rounding                       |            |        |        | per NFR-03, and unit test splits that do not divide evenly                                                                                                |
| Concurrent actions produce inconsistent balances | Data       | Low    | Medium | Validate on the server that a settlement cannot exceed the outstanding balance, and enforce integrity through database constraints                        |
| Merge conflicts                                  | Technical  | Medium | Medium | Use short-lived feature branches, merge through pull requests, and keep branches current with main                                                        |
| Authentication or authorization vulnerabilities  | Security   | Medium | High   | Use established authentication practices, hash passwords with bcrypt, enforce household scoping on every request, and validate all input on the server    |
| Application defects                              | Technical  | Medium | Medium | Implement automated unit and integration tests, run them in continuous integration, and perform manual testing throughout development                     |
| Deployment problems                              | Deployment | Medium | High   | Deploy a basic version well before the final milestone rather than at the end, and containerize the application so local and deployed environments match  |
| Free hosting tier limitations                    | Deployment | Medium | Low    | Warm the application before a demonstration to avoid cold-start delay, and evaluate a low-cost paid tier if the free database tier proves insufficient    |
| GitHub or Render unavailable near a deadline     | Deployment | Low    | Medium | The application runs locally through Docker Compose independently of either service, and local instructions are maintained alongside the deployed version |
| Data loss                                        | Data       | Low    | High   | Use hosting-provider database backup capabilities where available, and maintain seed data so a working database can be reconstructed                      |
| Requirements change significantly                | Scheduling | Low    | Medium | Follow Agile prioritization while holding to the original MVP scope, and keep changes                                                                     |

|  |  |  |  |                                       |
|--|--|--|--|---------------------------------------|
|  |  |  |  | small as the course guidance requires |
|--|--|--|--|---------------------------------------|

The two most significant risks to this project are scope expansion and the availability of a two-person team. Scope expansion is managed by treating the MVP as fixed and every additional feature as optional until the required functionality is complete. The team availability risk is managed structurally rather than reactively: because both members work across the full stack and review all of each other's work, neither member holds exclusive knowledge of any part of the application, so the project can continue if one member is temporarily unavailable.

## 12. SUCCESS CRITERIA

RoomSync will be considered successful if the following conditions are met:

- SC-01 Account access. A new user can register, log in, and securely access their account.
- SC-02 Household management. A user can create or join a household and view its members.
- SC-03 Expense management. A household member can create a shared expense and select participants.
- SC-04 Accurate expense splitting. The system correctly calculates equal, custom, and percentage-based expense shares, distributes any rounding remainder so that shares sum exactly to the total, and rejects invalid splits.
- SC-05 Balance tracking. Users can view the amounts they owe and the amounts owed to them, derived from recorded expenses and settlements.
- SC-06 Settlement recording. A household member can record that a debt has been paid, and the outstanding balance between the two members is reduced by that amount.
- SC-07 Chore management. Users can create, assign, and complete household chores.
- SC-08 Dashboard. Users can access their current balances, assigned chores, household members, and recent activity through a centralized dashboard.
- SC-09 Security. Users cannot read or modify household information for households to which they do not belong, and this restriction is enforced on the server for every request.
- SC-10 Testing. Automated unit and integration tests cover the expense splitting, balance calculation, and authorization logic, and the full suite passes in continuous integration on every pull request.
- SC-11 Independent execution. The TA can clone the repository and run the application on an independent computer at each required milestone using the documented setup instructions.
- SC-12 Deployment. The final application is deployed and accessible at a public URL.
- SC-13 Maintainability. Another developer can clone the repository and run the application locally using only the documented setup instructions, without assistance from

the team, and the repository contains organized source code, documentation, version history, and issue tracking.

The final application will be evaluated against these criteria at the milestone where each becomes applicable. The functional criteria, SC-01 through SC-08, will be verified through a documented manual walkthrough of the corresponding user stories using the demonstration accounts and seed data provided in the repository, and through the acceptance criteria defined in the requirements section. The calculation and authorization criteria, SC-04, SC-05, SC-06, and SC-09, will additionally be verified by automated tests, so that correctness is demonstrated rather than asserted. SC-10 will be evaluated from the coverage report and the continuous integration results. SC-11, SC-12, and SC-13 will be evaluated by following the repository's verification guide from a clean environment, which is the same process the TA uses to run the submitted version.

## 13. REFERENCES

The following resources have been consulted or are expected to be consulted during development. All sources accessed September 2, 2026.

### **Frameworks and libraries**

- [1] React Documentation, Meta Platforms, Inc. <https://react.dev>
- [2] TypeScript Documentation, Microsoft. <https://www.typescriptlang.org/docs>
- [3] Node.js API Documentation, OpenJS Foundation. <https://nodejs.org/api>
- [4] Express Documentation, OpenJS Foundation. <https://expressjs.com>
- [5] PostgreSQL Documentation, PostgreSQL Global Development Group. <https://www.postgresql.org/docs>
- [6] Prisma Documentation, Prisma Data, Inc. <https://www.prisma.io/docs>
- [7] bcrypt, npm package documentation. <https://www.npmjs.com/package/bcrypt>

### **Tools and services**

- [8] GitHub Documentation, GitHub, Inc. <https://docs.github.com>
- [9] GitHub Actions Documentation, GitHub, Inc. <https://docs.github.com/actions>
- [10] Docker Documentation, Docker, Inc. <https://docs.docker.com>
- [11] Render Documentation, Render. <https://render.com/docs>

## Testing

[12] Vitest Documentation, Vitest. <https://vitest.dev>

[13] SuperTest, npm package documentation. <https://www.npmjs.com/package/supertest>

## Security and accessibility

[14] OWASP Top 10, Open Worldwide Application Security Project. <https://owasp.org/www-project-top-ten>

[15] Web Content Accessibility Guidelines (WCAG) 2.2, World Wide Web Consortium. <https://www.w3.org/TR/WCAG22>

## Competing products

[16] Splitwise, Splitwise, Inc. <https://www.splitwise.com>

[17] Venmo, PayPal, Inc. <https://venmo.com>

[18] Tody, Looploop ApS. <https://todyapp.com>

Additional sources, competing applications, libraries, and services will be added to this section as they are incorporated into the project.

## CONTRIBUTION STATEMENTS:

Agustin Lemuz-Juarez: Drafted the executive summary, proposed application, MVP scope, initial design and architecture, and risks and mitigation sections, and produced the workflow, architecture, and entity relationship diagrams.

Orlando Rodriguez Valdez: Drafted the target users and personas, initial requirements, data and external resources, agile development process, and milestone plan sections, and compiled the references. Set up the shared GitHub repository and assembled the initial README and prioritized backlog.

Team responsibilities and success criteria were written jointly, and both members reviewed the full document for consistency before submission.

README.md:

# RoomSync

RoomSync is a web-based household management application that helps roommates coordinate shared expenses, chores, and household membership in one place. Unlike trip-based expense splitters, RoomSync is built around a **standing household**: balances persist and carry forward across the length of a lease instead of resetting, and expenses and chores are managed in the same shared space.

This repository contains the source code, documentation, and project artifacts for RoomSync, developed as part of a Software Design and Development course project.

## Status

This repository is at **Milestone 0 (Propose)**. No application code has been written yet. This milestone establishes the project proposal, architecture, backlog, and repository structure that later milestones will build on.

| Milestone   | Goal                      | Status      |
|-------------|---------------------------|-------------|
| Milestone 0 | Propose                   | In progress |
| Milestone 1 | Build the MVP             | Not started |
| Milestone 2 | Test, improve, and secure | Not started |
| Milestone 3 | Deploy                    | Not started |

## Team

| Name                     | Email                                                                          | Role                                               |
|--------------------------|--------------------------------------------------------------------------------|----------------------------------------------------|
| Orlando Rodriguez Valdez | <a href="mailto:orodriguezval@crimson.ua.edu">orodriguezval@crimson.ua.edu</a> | Accounts, auth/session, balances, settlements      |
| Agustin Lemuz-Juarez     | <a href="mailto:alemuzjuarez@crimson.ua.edu">alemuzjuarez@crimson.ua.edu</a>   | Households, invitations, expense entry/split logic |

## Planned tech stack

| Layer            | Technology                                  |
|------------------|---------------------------------------------|
| Frontend         | React + TypeScript                          |
| Backend          | Node.js + Express + TypeScript              |
| Database         | PostgreSQL                                  |
| ORM              | Prisma                                      |
| Auth             | Session-based auth, bcrypt password hashing |
| Containerization | Docker + Docker Compose                     |
| CI/CD            | GitHub Actions                              |
| Testing          | Vitest or Jest + Supertest                  |
| Deployment       | Render                                      |

See the [project proposal](#) for full justification of these choices.

## Repository structure (planned)

```
roomsync/  
├─ client/           # React frontend  
├─ server/           # Node/Express API  
├─ prisma/           # Database schema and migrations  
├─ docs/             # Proposal, diagrams, design artifacts  
├─ .github/          # GitHub Actions workflows  
└─ docker-compose.yml
```

This structure will be populated starting at Milestone 1.

## Getting started

Setup and run instructions will be added at Milestone 1, once the application is runnable end to end. At that point this section will include:

- Prerequisites (Node version, Docker)
- Environment variable setup (`.env.example`)
- Database migration and seed instructions
- How to run the client and server locally
- Demonstration account credentials

## Documentation

- [Project proposal](#)
- [Prioritized backlog](#)
- Architecture diagram — see proposal, Figure 2
- Entity relationship diagram — see proposal, Figure 3

## Contributing

All work happens on feature branches off `main` and is merged through pull requests requiring review from the other team member. See the proposal's Agile Development Process section for branching conventions, commit message format, and the Definition of Done.

## License

Course project — not currently licensed for outside use.

BACKLOG.md:

## RoomSync product backlog

This backlog lists all user stories identified in the Milestone 0 proposal, ordered by priority. **P0** stories are required for the Milestone 1 MVP. **P1** stories are planned but deferred until P0 work is complete. Stories will be entered as individual GitHub Issues, labeled by priority and area, and assigned to an owner at the start of each sprint.

Each story below includes its planned owner (per the proposal's team responsibilities) and a suggested sprint. Ownership and sprint assignment will be revisited at each sprint boundary as described in the proposal.

## P0 — Required for Milestone 1 MVP

| #     | Story                                                                                                                            | Area      | Owner   | Sprint   |
|-------|----------------------------------------------------------------------------------------------------------------------------------|-----------|---------|----------|
| US-01 | As a new user, I want to create a RoomSync account so that I can access the application.                                         | Account   | Orlando | Sprint 1 |
| US-02 | As a registered user, I want to log into RoomSync so that I can access my household information.                                 | Account   | Orlando | Sprint 1 |
| US-03 | As a user, I want to create a household so that I can manage responsibilities with my roommates.                                 | Household | Agustin | Sprint 1 |
| US-04 | As a household owner, I want to invite another user so that they can join my household.                                          | Household | Agustin | Sprint 2 |
| US-05 | As a household member, I want to record a shared expense so that the household can track who owes money.                         | Expense   | Agustin | Sprint 2 |
| US-06 | As a household member, I want to split an expense so that each roommate's financial responsibility is calculated correctly.      | Expense   | Agustin | Sprint 2 |
| US-07 | As a user, I want to see what I owe and what others owe me so that I understand my household balance.                            | Expense   | Orlando | Sprint 2 |
| US-08 | As a household member, I want to record that a debt has been paid so that balances reflect money that has already changed hands. | Expense   | Orlando | Sprint 2 |
| US-09 | As a household member, I want to create and assign chores so that household responsibilities are distributed among roommates.    | Chore     | Agustin | Sprint 2 |

|       |                                                                                                                   |           |         |          |
|-------|-------------------------------------------------------------------------------------------------------------------|-----------|---------|----------|
| US-10 | As a household member, I want to view a household dashboard so that I can quickly understand my responsibilities. | Dashboard | Orlando | Sprint 2 |
|-------|-------------------------------------------------------------------------------------------------------------------|-----------|---------|----------|

## P1 — Planned for MVP, deferred until P0 is complete

| #     | Story                                                                                                                                                                             | Area    | Owner   | Sprint   |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|---------|----------|
| US-11 | As a household member, I want to view past expenses, settlements, and chores so that I can check what a charge was for, confirm a debt was paid, or confirm a task was completed. | History | Orlando | Sprint 3 |

## Backlog by area

Grouping the same stories by feature area, for quick reference when planning a sprint around a single part of the system:

- **Account management:** US-01, US-02
- **Household management:** US-03, US-04
- **Expense management:** US-05, US-06, US-07, US-08
- **Chore management:** US-09
- **Dashboard:** US-10
- **History:** US-11

## Future features (not in backlog yet)

Identified in the proposal as potential post-MVP work, to be turned into backlog items if time permits after Milestone 2: recurring expenses, recurring chores, email/push notifications, expense categories, expense editing and deletion, member removal, support for multiple households per user, calendar integration, advanced household analytics, and a native mobile application.

## Explicitly out of scope

Not planned for this project at any milestone: actual payment processing, integration with Venmo/Cash App/Zelle/banking services, native mobile applications (beyond the future-features

list above), real-time household chat, automatic bank transaction importing, AI-based recommendations, and automatic bill retrieval from utility providers.

## Definition of Done

A story is not moved to Done until:

- The implementation meets its acceptance criteria (see the proposal's Initial Requirements section).
- The implementation follows project coding standards.
- Relevant unit or integration tests are written and passing.
- Continuous integration checks pass.
- The change has been reviewed and approved by the other team member.
- The change integrates without breaking existing functionality.
- No known critical defects remain.
- Related documentation, including the README, is updated.